数据结构基础 (FDS) 期中核心复习讲义 (含代码详解)

根据 FDS 课程 PPT 整理

2026 年 4 月 27 日

目录

1 第一部分：算法分析基础 (Algorithm Analysis) 3

1.1 1. 算法的定义与五大特征 3

1.2 2. 最大子列和问题核心代码 3

2 第二部分：线性表、栈与队列 (Lists, Stacks, Queues) 4

2.1 1. 线性表 (List ADT) 的链表实现 4

2.2 2. 栈 (Stack ADT) 的数组实现 4

2.3 3. 队列 (Queue ADT) 的循环数组实现 5

3 第三部分：二叉树与二叉搜索树 (Binary Trees & BST) 7

3.1 1. 二叉树的四种遍历 (Tree Traversals) 7

3.1.1 前序、中序、后序遍历 (递归实现) 7

3.1.2 层序遍历 (Level-order Traversal) 7

3.2 2. BST 的查找操作 (Find) 8

3.3 3. BST 查找最大值与最小值 (FindMin & FindMax) 8

3.4 4. BST 的插入操作 (Insert) 9

3.5 5. BST 的删除操作 (Delete) 9

4 第四部分：优先队列 (Priority Queues / Heaps) 11

4.1 1. 插入 (Insert) - 上滤操作 11

4.2 2. 删除最小元 (DeleteMin) - 下滤操作 11

4.3 3. 线性建堆 (BuildHeap) ——$O(N)$ 实现 12

5 第五部分：并查集 (The Disjoint Set ADT) 13

5.1 1. 初始化 13

5.2 2. 基础查找与合并 (Basic Find & Union) 13

5.3 3. 按秩合并 (Union by Height) 13

5.4 4. 按大小合并 (Union by Size) 14

5.5 5. 路径压缩查找 (递归版) 14

5.6 6. 路径压缩查找 (迭代版) 14

6 第六部分：线段树 (Segment Tree) 16

6.1 1. 建树 (Build) 16

6.2 2. 区间查询 (Query) 16

6.3 3. 单点更新 (Point Update) 16

1 第一部分：算法分析基础 (Algorithm Analysis)

1.1 1. 算法的定义与五大特征

算法是为了完成特定任务而规定的一组有限指令集。必须满足以下五个准则：

- **输入 (Input)**：外部提供的零个或多个量。
- **输出 (Output)**：至少产生一个量。
- **确定性 (Definiteness)**：每条指令清晰无歧义。
- **有限性 (Finiteness)**：算法在执行有限步后必须终止。
- **有效性 (Effectiveness)**：每条指令足够基本，原则上可用纸笔计算完成。

1.2 2. 最大子列和问题核心代码

在线处理 (Online Algorithm) 算法，时间复杂度 $O(N)$。

```
int MaxSubsequenceSum(const int A[], int N) {
    int ThisSum, MaxSum, j;
    ThisSum = MaxSum = 0;
    for(j = 0; j < N; j++) {
        ThisSum += A[j];
        if(ThisSum > MaxSum)
            MaxSum = ThisSum;
        else if(ThisSum < 0)
            ThisSum = 0;
    }
    return MaxSum;
}
```

2 第二部分：线性表、栈与队列 (Lists, Stacks, Queues)

2.1 1. 线性表 (List ADT) 的链表实现

```c
typedef struct Node *PtrToNode;
struct Node {
    ElementType Element;
    PtrToNode Next;
};
typedef PtrToNode List;
typedef PtrToNode Position;

/* 插入操作 */
void Insert(ElementType X, List L, Position P) {
    Position TmpCell;
    TmpCell = malloc(sizeof(struct Node));
    if(TmpCell == NULL) FatalError("Out of space!!!");
    TmpCell->Element = X;
    TmpCell->Next = P->Next;
    P->Next = TmpCell;
}

/* 查找前驱节点操作 */
Position FindPrevious(ElementType X, List L) {
    Position P = L;
    while(P->Next != NULL && P->Next->Element != X)
        P = P->Next;
    return P;
}

/* 删除操作 */
void Delete(ElementType X, List L) {
    Position P, TmpCell;
    P = FindPrevious(X, L); /* 首先找到要删除节点的前驱 */

    /* 如果 P->Next 不为空，说明找到了 X */
    if(P->Next != NULL) {
        TmpCell = P->Next;          /* 标记要删除的节点 */
        P->Next = TmpCell->Next;    /* 越过被删除节点，重新连线 */
        free(TmpCell);              /* 释放内存 */
    }
}
```

2.2 2. 栈 (Stack ADT) 的数组实现

```
struct StackRecord {
    int Capacity;
    int TopOfStack;
    ElementType *Array;
};

void Push(ElementType X, Stack S) {
    if(IsFull(S))
        Error("Full stack");
    else
        S->Array[++S->TopOfStack] = X;
}

ElementType Pop(Stack S) {
    if(IsEmpty(S))
        Error("Empty stack");
    else
        return S->Array[S->TopOfStack--];
}
```

2.3 3. 队列 (Queue ADT) 的循环数组实现

在数组实现中，为了避免假溢出，通常采用循环数组结构。入队和出队时，指针的移动需要使用取余运算 % Capacity。

```
struct QueueRecord {
    int Capacity;
    int Front;
    int Rear;
    int Size;
    ElementType *Array;
};
typedef struct QueueRecord *Queue;

/* 入队操作 (Enqueue) */
void Enqueue(ElementType X, Queue Q) {
    if(IsFull(Q)) {
        Error("Full queue");
    } else {
        Q->Size++;
        Q->Rear = (Q->Rear + 1) % Q->Capacity; /* 循环游标的关键操作 */
        Q->Array[Q->Rear] = X;
    }
}

```

```
/* 出队操作 (Dequeue) */
ElementType Dequeue(Queue Q) {
    ElementType FrontElem;
    if(IsEmpty(Q)) {
        Error("Empty queue");
        return 0; /* 根据具体类型返回错误标志 */
    } else {
        Q->Size--;
        FrontElem = Q->Array[Q->Front];
        Q->Front = (Q->Front + 1) % Q->Capacity; /* 循环游标的关键操作 */
        return FrontElem;
    }
}
```

3 第三部分：二叉树与二叉搜索树 (Binary Trees & BST)

二叉搜索树 (BST) 性质：对任意节点，左子树所有节点的值 < 该节点的值 < 右子树所有节点的值。

3.1 1. 二叉树的四种遍历 (Tree Traversals)

遍历是按照一定规则访问树中所有节点且仅访问一次的过程。

3.1.1 前序、中序、后序遍历 (递归实现)

```c
/* 前序遍历 (Preorder): 根 -> 左 -> 右 */
void PreOrderTraversal(SearchTree T) {
    if(T != NULL) {
        printf("%d␣", T->Element);
        PreOrderTraversal(T->Left);
        PreOrderTraversal(T->Right);
    }
}

/* 中序遍历 (Inorder): 左 -> 根 -> 右
 * 特别注意: 对 BST 进行中序遍历, 可以得到一个递增的有序序列!
 */
void InOrderTraversal(SearchTree T) {
    if(T != NULL) {
        InOrderTraversal(T->Left);
        printf("%d␣", T->Element);
        InOrderTraversal(T->Right);
    }
}

/* 后序遍历 (Postorder): 左 -> 右 -> 根 */
void PostOrderTraversal(SearchTree T) {
    if(T != NULL) {
        PostOrderTraversal(T->Left);
        PostOrderTraversal(T->Right);
        printf("%d␣", T->Element);
    }
}
```

3.1.2 层序遍历 (Level-order Traversal)

层序遍历也就是广度优先搜索 (BFS)，必须借助“队列”来实现。

```c
void LevelOrderTraversal(SearchTree T) {
    Queue Q;
```

```
    SearchTree Node;

    if(T == NULL) return;

    Q = CreateQueue(MaxElements); /* 假设已实现队列初始化 */
    Enqueue(T, Q);

    while(!IsEmpty(Q)) {
        Node = Dequeue(Q);
        printf("%d␣", Node->Element);

        /* 将左右子节点依次入队 */
        if(Node->Left) Enqueue(Node->Left, Q);
        if(Node->Right) Enqueue(Node->Right, Q);
    }
    DisposeQueue(Q);
}
```

3.2 2. BST 的查找操作 (Find)

时间复杂度：平均 $O(\log N)$，最坏情况退化为链表 $O(N)$。

```
Position Find(ElementType X, SearchTree T) {
    if(T == NULL)
        return NULL;
    if(X < T->Element)
        return Find(X, T->Left);
    else if(X > T->Element)
        return Find(X, T->Right);
    else
        return T; /* Match */
}
```

3.3 3. BST 查找最大值与最小值 (FindMin & FindMax)

```
/* 递归实现：寻找最小值（一直向左走到底） */
Position FindMin(SearchTree T) {
    if(T == NULL)
        return NULL;
    else if(T->Left == NULL)
        return T;
    else
        return FindMin(T->Left);
}

```

```
/* 迭代实现：寻找最大值（一直向右走到底，不用递归可以节省栈空间）*/
Position FindMax(SearchTree T) {
    if(T != NULL)
        while(T->Right != NULL)
            T = T->Right;
    return T;
}
```

3.4 4. BST 的插入操作 (Insert)

```
SearchTree Insert(ElementType X, SearchTree T) {
    if(T == NULL) {
        /* 找到空位，创建并返回新节点 */
        T = malloc(sizeof(struct TreeNode));
        if(T == NULL) FatalError("Out of space!!!");
        T->Element = X;
        T->Left = T->Right = NULL;
    }
    else if(X < T->Element)
        T->Left = Insert(X, T->Left);
    else if(X > T->Element)
        T->Right = Insert(X, T->Right);
    /* Else X is in the tree already; we'll do nothing */
    return T;
}
```

3.5 5. BST 的删除操作 (Delete)

删除是 BST 中最复杂的操作，分为三种情况：1. **叶子节点**：直接删除。2. **只有一个孩子**：用其唯一的孩子节点取代它的位置。3. **有两个孩子**：找到右子树的最小值 (或左子树的最大值) 替换该节点的数据，然后递归删除那个最小 (最大) 节点。

```
SearchTree Delete(ElementType X, SearchTree T) {
    Position TmpCell;

    if(T == NULL)
        Error("Element not found");
    else if(X < T->Element)  /* 目标比当前节点小，去左子树删 */
        T->Left = Delete(X, T->Left);
    else if(X > T->Element)  /* 目标比当前节点大，去右子树删 */
        T->Right = Delete(X, T->Right);

    /* 找到了目标节点，且该节点有两个孩子 */
    else if(T->Left && T->Right) {
```

```
13          /* 在右子树中找最小节点来替换 */
14          TmpCell = FindMin(T->Right);
15          T->Element = TmpCell->Element;
16          /* 递归地在右子树中删除那个最小节点 */
17          T->Right = Delete(T->Element, T->Right);
18      }
19
20      /* 找到了目标节点，且该节点只有 1 个或 0 个孩子 */
21      else {
22          TmpCell = T;
23          if(T->Left == NULL)         /* 只有右孩子，或者没孩子 */
24              T = T->Right;
25          else if(T->Right == NULL) /* 只有左孩子 */
26              T = T->Left;
27          free(TmpCell);
28      }
29      return T;
30  }
```

4 第四部分：优先队列 (Priority Queues / Heaps)

以最小堆 (Min-Heap) 为例，结构为完全二叉树，且满足任何节点的值 ≤ 其所有子节点的值。

4.1 1. 插入 (Insert) - 上滤操作

```
void Insert(ElementType X, PriorityQueue H) {
    int i;
    if(IsFull(H)) {
        Error("Priority queue is full");
        return;
    }
    /* Percolate up */
    for(i = ++H->Size; H->Elements[i / 2] > X; i /= 2)
        H->Elements[i] = H->Elements[i / 2];
    H->Elements[i] = X;
}
```

4.2 2. 删除最小元 (DeleteMin) - 下滤操作

```
ElementType DeleteMin(PriorityQueue H) {
    int i, Child;
    ElementType MinElement, LastElement;

    if(IsEmpty(H)) {
        Error("Priority queue is empty");
        return H->Elements[0];
    }
    MinElement = H->Elements[1];
    LastElement = H->Elements[H->Size--];

    /* Percolate down */
    for(i = 1; i * 2 <= H->Size; i = Child) {
        Child = i * 2; /* Find smaller child */
        if(Child != H->Size && H->Elements[Child + 1] < H->Elements[Child])
            Child++;

        if(LastElement > H->Elements[Child]) /* Percolate one level */
            H->Elements[i] = H->Elements[Child];
        else
            break;
    }
    H->Elements[i] = LastElement;
```

```
    return MinElement;
}
```

4.3 3. 线性建堆 (BuildHeap) ——$O(N)$ 实现

线性建堆（Floyd 算法）不是通过多次调用插入来实现的（那样是 $O(N \log N)$），而是通过对所有非叶子节点进行“下滤”操作来实现。

```
/* 下滤 (PercolateDown) 辅助函数：将下标为 i 的元素向下移动到合适位置 */
void PercolateDown(PriorityQueue H, int i) {
    int Child;
    ElementType Tmp = H->Elements[i];

    for(; i * 2 <= H->Size; i = Child) {
        Child = i * 2;
        /* 1. 挑选较小的孩子 (注意：此处可增加“极大值哨兵”优化) */
        if(Child != H->Size && H->Elements[Child + 1] < H->Elements[Child])
            Child++;

        /* 2. 如果孩子比当前元素小，则把孩子拉上来 */
        if(Tmp > H->Elements[Child])
            H->Elements[i] = H->Elements[Child];
        else
            break;
    }
    H->Elements[i] = Tmp;
}

/* 线性建堆主函数 */
void BuildHeap(PriorityQueue H) {
    int i;
    /* 从最后一个非叶子节点 (Size/2) 开始，一直处理到根节点 (1) */
    for(i = H->Size / 2; i > 0; i--) {
        PercolateDown(H, i);
    }
}
```

复杂度分析 (Time Complexity): 虽然单次 `PercolateDown` 的最坏复杂度是 $O(\log N)$，但建堆的总复杂度是所有节点的高度之和。

- 底部节点多，但高度小（下滤距离短）。
- 顶部节点少，但高度大（下滤距离长）。
- 数学证明得出总高度和 $S = \sum Height(i) = N - \log N - 1 = O(N)$。

5 第五部分：并查集 (The Disjoint Set ADT)

使用数组实现森林结构，支持按秩合并和路径压缩。

5.1 1. 初始化

```
void Initialize(DisjSet S) {
    int i;
    for(i = N; i > 0; i--)
        S[i] = -1;
    /* -1 indicates a root with size/height 1 */
}
```

5.2 2. 基础查找与合并 (Basic Find & Union)

```
/* 基础查找 (Basic Find): 无路径压缩, 最坏时间复杂度 O(N) */
SetType FindBasic(ElementType X, DisjSet S) {
    if(S[X] <= 0)
        return X;
    else
        return FindBasic(S[X], S);
}

/* 基础合并 (Basic Union): 直接合并, 最坏可能导致退化 */
void UnionBasic(DisjSet S, SetType Root1, SetType Root2) {
    /* 默认将 Root2 直接挂载到 Root1 下 */
    S[Root2] = Root1;
}
```

5.3 3. 按秩合并 (Union by Height)

```
void SetUnion(DisjSet S, SetType Root1, SetType Root2) {
    if(S[Root2] < S[Root1]) /* Root2 is deeper */
        S[Root1] = Root2;
    /* Make Root2 new root */
    else {
        if(S[Root1] == S[Root2]) /* Same height */
            S[Root1]--;
        /* Update height */
        S[Root2] = Root1;
        /* Make Root1 new root */
    }
}
```

5.4 4. 按大小合并 (Union by Size)

```
/* 按大小合并 (Union by Size) */
void SetUnionBySize(DisjSet S, SetType Root1, SetType Root2) {
    /* 注意: S[Root] 是负数, 其绝对值表示集合大小 */
    if(S[Root2] < S[Root1]) {
        /* Root2 的值更小, 说明 Root2 的实际 Size 更大 */
        S[Root2] += S[Root1]; /* 更新 Root2 的总 Size */
        S[Root1] = Root2;     /* 将较小的 Root1 挂载到 Root2 上 */
    }
    else {
        /* Root1 的 Size 大于或等于 Root2 的 Size */
        S[Root1] += S[Root2]; /* 更新 Root1 的总 Size */
        S[Root2] = Root1;     /* 将较小的 Root2 挂载到 Root1 上 */
    }
}
```

5.5 5. 路径压缩查找 (递归版)

```
SetType Find(ElementType X, DisjSet S) {
    if(S[X] <= 0)
        return X;
    else
        return S[X] = Find(S[X], S);
}
```

5.6 6. 路径压缩查找 (迭代版)

```
/* 路径压缩查找 —— 迭代实现版本 */
SetType FindIterative(ElementType X, DisjSet S) {
    ElementType curr = X;
    ElementType root, trail;

    /* 第一遍扫描: 向上遍历找到根节点 */
    while(S[curr] > 0) {
        curr = S[curr];
    }
    root = curr;

    /* 第二遍扫描: 进行路径压缩, 将路径上的节点都直接挂载到根节点 */
    curr = X;
    while(curr != root) {
        trail = S[curr]; /* 暂存当前节点的原本父节点 */
        S[curr] = root;  /* 当前节点直接指向根节点 */
```

```
17          curr = trail;     /* 沿着原本的路径继续向上处理 */
18      }
19
20      return root;
21  }
```

6 第六部分：线段树 (Segment Tree)

用于解决频繁的数组区间查询。

6.1 1. 建树 (Build)

```
void build(int node, int start, int end) {
    if(start == end) {
        tree[node] = A[start];
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }
}
```

6.2 2. 区间查询 (Query)

```
int query(int node, int start, int end, int L, int R) {
    if(R < start || end < L) return 0; /* Outside range */
    if(L <= start && end <= R) return tree[node]; /* Inside range */

    int mid = (start + end) / 2;
    int p1 = query(2 * node, start, mid, L, R);
    int p2 = query(2 * node + 1, mid + 1, end, L, R);
    return p1 + p2;
}
```

6.3 3. 单点更新 (Point Update)

将数组中下标为 idx 的元素增加 val，并自底向上更新线段树节点。

```
void update(int node, int start, int end, int idx, int val) {
    if(start == end) {
        /* 找到叶子节点，更新其值 */
        A[idx] += val;
        tree[node] += val;
    } else {
        int mid = (start + end) / 2;
        if(start <= idx && idx <= mid) {
            /* 如果目标在左半区间，递归左子树 */
            update(2 * node, start, mid, idx, val);
        } else {
            /* 如果目标在右半区间，递归右子树 */
```

```
13              update(2 * node + 1, mid + 1, end, idx, val);
14          }
15          /* 子节点更新完毕, 自底向上更新当前节点 */
16          tree[node] = tree[2 * node] + tree[2 * node + 1];
17      }
18  }
```